🔬 FE 코드 보강 자료 — 치트시트 갭 보완

> 기존 **아키텍처 치트시트**(개념 위주) + **FE 코드 자료**(프론트 컴포넌트 위주)에서 **코드 근거가 부족한 5개 영역**을 보강.

---

1. 보안 입력 검증 — security/policy.ts

치트시트에서 "보안 레이어" "속도 제한"을 텍스트로만 설명. 실제 **입력 단계에서 악의적 패턴을 어떻게 잡는지** 코드 근거가 없음.

1-1. Base64 위장 텍스트 탐지

```
function looksLikeBase64Payload(input: string): boolean {
    const compact = input.replace(/\s+/g, "");
    if (compact.length < 256) return false;
    if (compact.length % 4 !== 0) return false;
    return BASE64_ONLY_REGEX.test(compact);
}
```

Q: 이 검증이 왜 필요한가?

> 공격자가 텍스트 입력 필드에 **이미지 Base64를 직접 붙여넣어** LLM 토큰을 대량 소모시키는 공격을 막는다. 256자 미만은 정상적인 짧은 영문 입력일 수 있어 무시하고, 4의 배수 + Base64 문자만으로 구성된 경우에만 차단.

1-2. 이미지 페이로드 이중 검증

```
// 1차: MIME 타입 화이트리스트
if (!PARSE_INPUT_POLICY.allowedImageMimeTypes.includes(normalizedMimeType))
// 2차: 바이트 크기 → 8MB 제한
if (input.byteLength > PARSE_INPUT_POLICY.maxImageBytes)
// 3차: Base64 인코딩 길이 → 3MB 제한
if (input.base64Length > PARSE_INPUT_POLICY.maxImageBase64Length)
// 4차: Base64 실제 디코딩 검증
if (!isStrictBase64Payload(input.base64Payload))  // Buffer.from(x,
"base64").toString("base64") === x
```

Q: 바이트 크기랑 Base64 길이를 왜 따로 체크하나?

> 클라이언트가 보낸 `byteLength` 는 **자기 신고 값**이라 위조가 가능하다. Base64 인코딩 후 실제 길이(`base64Length`)를 서버에서 독립적으로 계산해 교차 검증. 4차의 isStrictBase64Payload은 **디코딩→재인코딩이 동일한지** 확인하여 손상된/위조된 데이터를 걸러낸다.

1-3. HMAC 기반 식별자 해싱

```
function hashSecurityValue(value: string, namespace = "general"): string {
    const digest = createHmac("sha256", getSecuritySecret())
        .update(`${SECURITY_DIGEST_VERSION}:${namespace}:`)
        .update(value)
        .digest("hex");
    return `${namespace}:${SECURITY_DIGEST_VERSION}:${digest}`;
}
```

Q: 단순 SHA-256이 아닌 HMAC을 쓰는 이유?

> SHA-256은 **비밀키 없이 누구나 동일 해시를 생성**할 수 있어, DB가 유출되면 레인보우 테이블로 원본 IP를 역추적할 수 있다. HMAC은 서버만 아는 `BETTER_AUTH_SECRET` 을 키로 사용하므로, DB가 유출돼도 원본 IP를 알아낼 수 없다.

> **namespace prefixing** (`ip:v1:abc...`)으로 같은 값이라도 용도(IP/세션/유저)별 해시가 달라져 **cross-scope 매칭을 원천 차단**.

1-4. IPv6 서브넷 정규화

```
function normalizeIpv6Subnet(value: string): string | null {
    // ::ffff:192.168.0.1 → 순수 IPv6로 변환
    // 전체 8개 블록 확장 → 앞 4블록만 사용 (/64 서브넷)
    return `${expanded.slice(0, 4).join(":")}::/64`;
}
```

Q: /64 서브넷으로 정규화하는 이유?

> IPv6는 같은 사용자가 ISP에 의해 **수시로 다른 주소를 받을 수** 있다. 개별 주소로 rate limit을 걸면 무의미하다. /64 서브넷(상위 64비트)은 가정/조직 단위에 할당되므로, 같은 네트워크의 요청을 하나로 묶어 의미있는 제한을 건다.

---

2. Provider 폴백 + 쿨다운 — parse-core.ts

치트시트에서 "3-Provider 전략"을 테이블로만 설명. **실제 폴백/쿨다운 코드 흐름**이 없음.

2-1. 인메모리 이미지 사용량 추적

```
const imageFireworksUsageMap = new Map<string, {
    count: number;           // 성공 응답 누적
    lastUsed: number;        // 마지막 사용 시각
    blockedUntil?: number;   // 쿨다운 해제 시각
}>();
```

Q: DB가 아닌 인메모리인 이유? 한계는?

> 이 카운터는 **비용 최적화용**이지 보안 제한이 아니다. 서버 재시작 시 리셋되어도 Fireworks 무료 티어 3회를 다시 쓰는 것뿐이라 실해가 없다. 보안 rate limit은 별도로 DB에서 관리(security/index.ts).

> **메모리 누수 방지:** `MAX_MAP_SIZE = 1000` 에 도달하면 24시간 이상 미사용 엔트리를 정리 (pruneStaleImageFireworksEntries).

2-2. "복구 가능한 실패"만 폴백

```
function isRecoverableProviderFailure(message: string): boolean {
    const normalized = message.toLowerCase();
    return normalized.includes("llm 응답 시간 초과")
        || normalized.includes("fetch failed")
        || normalized.includes("500") || normalized.includes("503")
        // ... 네트워크/서버 오류
    ;
}
```

Q: 왜 모든 실패에 폴백하지 않나?

> "가계부와 관련 없는 입력"(OOD) 같은 **콘텐츠 오류**는 다른 provider로 보내도 같은 결과다. 네트워크/타임아웃/서버 오류만 **인프라 문제**이므로 다른 provider가 성공할 가능성이 있다. 불필요한 폴백은 비용 낭비 + 응답 지연.

2-3. 장애 시 10분 쿨다운

```
function activateImageFireworksCooldown(sessionId: string, reason: string): void {
    // Fireworks가 복구 가능한 오류로 실패하면
    // 해당 세션에서 10분간 Fireworks를 건너뛰고 바로 Kimi를 사용
    blockedUntil = Date.now() + IMAGE_FIREWORKS_FAILURE_COOLDOWN_MS;  // 10분
}
```

Q: 왜 세션 단위 쿨다운인가?

> 전역 쿨다운을 걸면 **Fireworks가 특정 이미지에서만 실패해도** 모든 사용자가 영향받는다. 세션 단위로 쿨다운을 걸면 장애가 발생한 사용자만 빠르게 Kimi로 전환되고, 다른 사용자는 계속 Fireworks를 사용한다.

2-4. 입력 길이별 타임아웃

```
≤100자 → 45초 ("CU 3500" 같은 단순 입력)
≤400자 → 70초 (은행 메시지 등 중간 길이)
>400자 → 100초 (여러 거래 포함된 긴 입력)
```

Q: 왜 고정 타임아웃이 아닌가?

> 짧은 입력에 100초를 기다리게 하면 사용자가 "멈춘 건가?" 하고 재시도한다. **입력 복잡도에 비례하는 기대치**를 반영해 UX와 안정성 균형.

---

3. LLM 클라이언트 팩토리 — client.ts

치트시트에서 "OpenAI SDK 통일, 싱글턴 캐시" 한 줄. 실제 패턴이 없음.

```
// baseURL만 바꿔서 3개 벤더를 동일 인터페이스로 호출
const configs: Record<LLMProvider, () => LLMConfig> = {
    minimax: () => ({
        client: new OpenAI({ baseURL: "https://api.minimax.io/v1" }),
        model: "MiniMax-M2.5",
        extra_body: { reasoning_split: true },
    }),
    kimi: () => ({
        client: new OpenAI({ baseURL: "https://api.moonshot.ai/v1" }),
        model: "kimi-k2.5",
        extra_body: { chat_template_kwargs: { thinking: false } },
    }),
    fireworks: () => ({
        client: new OpenAI({ baseURL: "https://api.fireworks.ai/inference/v1" }),
        model: "accounts/fireworks/models/kimi-k2p5",
    }),
};

// 지연 초기화 + 싱글턴 캐시
const cache = new Map<LLMProvider, LLMConfig>();
export function getLLMConfig(provider?: LLMProvider): LLMConfig {
    const resolved = provider || (process.env.LLM_PROVIDER as LLMProvider) || "kimi";
    if (cache.has(resolved)) return cache.get(resolved)!;
    const config = configs[resolved]();
    cache.set(resolved, config);
    return config;
}
```

Q: 왜 팩토리 패턴인가?

> 새 provider(예: Claude, Gemini)를 추가하려면 `configs` 객체에 **3줄만 추가**하면 된다. 호출 코드(parse-core.ts, index.ts)는 수정할 필요 없음. **개방-폐쇄 원칙(OCP)** 적용.

Q: `extra_body` 가 뭔가?

> OpenAI SDK에 없는 **벤더 고유 파라미터**를 전달하는 관례적 필드. MiniMax의 `reasoning_split: true` (추론 과정과 답변 분리), Kimi의 `thinking: false` (CoT 비활성화로 토큰 절감) 등.

---

4. 인증 DB 훅 + 세션 최소화 — auth.ts

치트시트에서 "databaseHooks로 카테고리 시딩, IP/UA 최소화" 설명. **실제 코드 패턴**이 없음.

4-1. 세션 생성 시 개인정보 최소화

```
session: {
    create: {
        before: async (session) => ({
            data: {
                ...session,
                ipAddress: minimizeSessionIpAddress(session.ipAddress),
                userAgent: minimizeSessionUserAgent(session.userAgent),
            },
        }),
    },
    update: {
        before: async (session) => ({
            // 세션 갱신 시에도 동일하게 최소화
    }),
},
```

Q: IP와 User-Agent를 왜 해싱하나?

> **개인정보보호법/GDPR 대응.** IP 주소는 개인 식별 정보(PII)로 분류될 수 있다. 원본을 저장하면 데이터 유출 시 사용자 위치 추적이 가능하다. HMAC 해싱하면 **같은 IP의 세션을 연결할 수는 있지만**, 원본 IP를 역산할 수 없다.

> **`create.before` 와 `update.before` 모두에 적용한 이유:** Better Auth는 세션 갱신(`updateAge: 86400`)마다 새 IP/UA를 기록한다. update 훅을 빠뜨리면 갱신 시점에 원본이 저장된다.

4-2. Recoverable 세션 에러 처리

```
const RECOVERABLE_SESSION_ERROR_MESSAGES = new Set([
    "Invalid token",
    "Session expired. Re-authenticate to perform this action.",
]);

export async function getServerSession() {
    try {
        return await auth.api.getSession({ headers: await headers() });
    } catch (error) {
        if (isRecoverableSessionError(error)) return null;  // 만료 → null (로그인 유도)
        throw error;  // 예상 못한 인증 오류 → 바로 전파
    }
}
```

Q: 왜 에러를 삼키는 건가?

> "세션 만료"는 **정상적인 라이프사이클**이다. 에러로 던지면 Error Boundary까지 올라가서 사용자에게 500 페이지가 보인다. `null` 을 반환하면 호출자(layout.tsx)가 **로그인 리다이렉트**로 우아하게 처리할 수 있다.

> **하지만 예상 못한 에러(DB 연결 실패 등)는 반드시 throw.** "삼키는" 범위를 `Set` 으로 명시해 범위를 제한.

---

5. LLM 응답 파싱 파이프라인 — llm/index.ts

치트시트에서 "JSON 파싱", "검증" 텍스트로만 설명. **실제 4단계 파이프라인 코드**가 없음.

5-1. `<think>` 태그 제거 (MiniMax 대응)

```
function stripReasoningBlocks(text: string): string {
    return text.replace(/<think>[\s\S]*?<\/think>\s*/gi, "").trim();
}
```

Q: 왜 LLM 응답에 HTML 태그가 섞이나?

> MiniMax M2.5는 `reasoning_split: true` 설정에도 불구하고, **간헐적으로** `<think>여기서 분석하면...</think>{ "intent": ... }` 형태로 응답한다. 이걸 제거하지 않으면 JSON 파싱이 실패한다. 모든 provider에 일괄 적용해도 부작용이 없으므로 방어적으로 처리.

5-2. JSON 추출 3단계 폴백

```
function extractJSON(text: string): string {
    const normalized = stripReasoningBlocks(text);
    // 1순위: ```json ... ``` 코드블록
    const fenced = normalized.match(/```(?:json)?\s*([\s\S]*?)```/);
    if (fenced) return fenced[1].trim();
    // 2순위: { ... } 객체 직접 추출
    const objectMatch = normalized.match(/\{[\s\S]*\}/);
    if (objectMatch) return objectMatch[0].trim();
    // 3순위: [ ... ] 배열 (하위 호환)
    const arrayMatch = normalized.match(/\[[\s\S]*\]/);
    if (arrayMatch) return arrayMatch[0].trim();
    return normalized.trim();
}
```

Q: LLM 응답 형식이 왜 이렇게 다양한가?

> **LLM은 비결정적이다.** 같은 프롬프트에도 마크다운 코드블록, 순수 JSON, 설명문+JSON을 섞어 반환한다. 프롬프트에서 "JSON만 반환하라"고 해도 100% 보장이 안 된다. 3단계 폴백으로 **어떤 형식으로 와도** JSON을 추출한다.

5-3. OOD 두 번째 필터 (LLM 레벨)

```
if (obj.rejected === true) {
    const reason = typeof obj.reason === "string"
        ? obj.reason
        : "가계부와 관련된 내용을 입력해 주세요.";
    throw new Error(reason);
}
```

> OOD 필터 1차(ood-filter.ts)를 통과한 **애매한 입력**도 LLM이 `{rejected: true}` 를 반환하면 여기서 최종 차단. **비용은 이미 발생했지만** 잘못된 데이터가 DB에 들어가는 것을 막는다.

5-4. 서버 측 AbortSignal 타임아웃

```
async function withTimeout<T>(task: (signal: AbortSignal) => Promise<T>, ms: number):
Promise<T> {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => {
        controller.abort(new LLMTimeoutError());
    }, ms);
    try {
        return await task(controller.signal);
    } finally {
        clearTimeout(timeoutId);
    }
}
```

Q: 클라이언트의 AbortController와 차이는?

	클라이언트 (NaturalInputBar)	서버 (llm/index.ts)
누가 취소	사용자가 직접 취소	서버가 타임아웃 시 자동 취소
무엇을 취소	`fetch(/api/parse)`	`fetch(LLM API)`
왜 필요	재입력 시 이전 요청 정리	LLM이 무한 대기하는 것 방지

> **서버의 abort는 "늦은 200 OK"를 방지**한다. 타임아웃 발생 후 LLM이 뒤늦게 응답해도, abort된 request는 이미 폐기되어 유령 응답이 처리되지 않는다.

---

6. 은행 메시지 전처리 — bank-message.ts

치트시트에서 "잔액/한도 등 노이즈 제거" 한 줄. 구체적 패턴이 없음.

```
const NOISE_PATTERNS = [
    /잔액\s*[:\s]?\s*[\d,]+원?/g,   // "잔액 1,234,567원" → 제거
    /누적\s*[:\s]?\s*[\d,]+원?/g,   // "누적 50,000원" → 제거
    /남은\s*한도\s*[:\s]?\s*[\d,]+원?/g,
    /일시불/g,                        // 결제 방식 정보 → 불필요
    /할부\s*\d+개?월?/g,             // "할부 3개월" → 불필요
];
```

Q: 왜 제거하나? LLM이 알아서 무시하면 안 되나?

> **LLM은 "잔액 1,234,567원"을 거래 금액으로 잘못 파싱할 수 있다.** 카드 승인 메시지의 실제 결제 금액은 9,000원인데, 잔액 1,234,567원을 금액으로 잡으면 완전히 틀린 결과가 된다. 전처리로 노이즈를 제거하면 LLM이 **정확한 금액에 집중**할 수 있다.

> **isBankMessage 판별이 선행:** 일반 텍스트에서는 이 전처리를 하지 않는다. "잔액"이라는 단어가 자산 등록 입력(`카카오뱅크 잔액 150만원`)에서는 핵심 정보이므로 제거하면 안 된다.

요약 — 치트시트와의 관계

갭	치트시트 설명	이 문서가 보강하는 것
보안 입력 검증	"보안 레이어" 테이블	Base64 탐지, HMAC, IPv6 정규화 **코드**
Provider 폴백	"3-Provider 전략" 표	쿨다운·복구 가능 판정·메모리 관리 **로직**
LLM 클라이언트	"OpenAI SDK 통일" 한 줄	팩토리 패턴· `extra_body` ·싱글턴 **구현**
인증 훅	DB 훅 개념 설명	세션 최소화·recoverable 에러 **패턴**
응답 파싱	"JSON 파싱" 한 줄	4단계 파이프라인· `<think>` 제거·OOD 2차 **구조**
은행 전처리	"노이즈 제거" 한 줄	정규식 패턴·판별 로직·LLM 정확도 연관 **근거**